# Homework 08

The former kernel-tuning question has moved to Homework 09.

Solutions for Questions 1 and 2 will be available after Monday, October 19, 2026. No solution can be provided for Question 3.

For Questions 1 and 2, use either R or Python.

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
library(knitr)
```

## Question 1: Boundary and margin

Use `margin-geometry-20.csv` from the combined ZIP, which contains 20 observations with predictors $x_1, x_2$ and class labels $y \in \{-1, 1\}$. Consider the score

$$f(x) = 0.8x_1 + 0.6x_2.$$

Since $\|(0.8, 0.6)\|_2 = 1$, the lines $f(x) = \pm 1$ are each a distance 1 from the decision boundary $f(x) = 0$.

```
d1 <- read_csv("data/margin-geometry-20.csv")
w <- c(0.8, 0.6)

d1 <- d1 |>
  mutate(
    f = w[1] * x1 + w[2] * x2,
    distance = abs(f),
    margin = y * f,
    status = case_when(
      abs(margin - 1) < 1e-8 ~ "on",
      margin < 1 ~ "inside",
      TRUE ~ "beyond"
    ),
    misclassified = margin < 0
  )
```

a. Calculate the distance from every observation to the decision boundary and its functional margin $yf(x)$. In a table, report these values and whether each observation is inside, on, or beyond the margin for its class. Compare $yf(x)$ with 1 to make this distinction.

Since the norm of (0.8, 0.6) is 1, the distance from an observation to the boundary is just |f(x)|. An observation is beyond the margin if yf(x) $>$ 1, on the margin if yf(x) $=$ 1, and inside the margin if yf(x) $<$ 1.

```
d1 |>
  select(observation, x1, x2, y, distance, margin, status) |>
  mutate(across(c(distance, margin), \(v) round(v, 3))) |>
  kable(col.names = c("Obs", "x1", "x2", "y", "Distance to boundary",
                      "Functional margin yf(x)", "Position"))
```

| Obs | x1 | x2 | y | Distance to boundary | Functional margin yf(x) | Position |
|---|---|---|---|---|---|---|
| 1 | 0.68 | 0.76 | 1 | 1.00 | 1.00 | on |
| 2 | 1.80 | -0.15 | 1 | 1.35 | 1.35 | beyond |
| 3 | 1.38 | 0.66 | 1 | 1.50 | 1.50 | beyond |
| 4 | 0.88 | 1.66 | 1 | 1.70 | 1.70 | beyond |
| 5 | 0.62 | 2.34 | 1 | 1.90 | 1.90 | beyond |
| 6 | 2.78 | -0.04 | 1 | 2.20 | 2.20 | beyond |
| 7 | 1.86 | 1.52 | 1 | 2.40 | 2.40 | beyond |
| 8 | 0.50 | 0.40 | 1 | 0.64 | 0.64 | inside |
| 9 | 0.10 | 0.20 | 1 | 0.20 | 0.20 | inside |
| 10 | -0.40 | 0.20 | 1 | 0.20 | -0.20 | inside |
| 11 | -0.92 | -0.44 | -1 | 1.00 | 1.00 | on |
| 12 | -0.50 | -1.50 | -1 | 1.30 | 1.30 | beyond |
| 13 | -1.90 | -0.05 | -1 | 1.55 | 1.55 | beyond |
| 14 | -0.44 | -2.33 | -1 | 1.75 | 1.75 | beyond |
| 15 | -1.98 | -0.61 | -1 | 1.95 | 1.95 | beyond |
| 16 | -2.74 | 0.07 | -1 | 2.15 | 2.15 | beyond |
| 17 | -1.90 | -1.55 | -1 | 2.45 | 2.45 | beyond |
| 18 | -0.50 | -0.40 | -1 | 0.64 | 0.64 | inside |
| 19 | 0.00 | -0.40 | -1 | 0.24 | 0.24 | inside |
| 20 | 0.40 | -0.20 | -1 | 0.20 | -0.20 | inside |

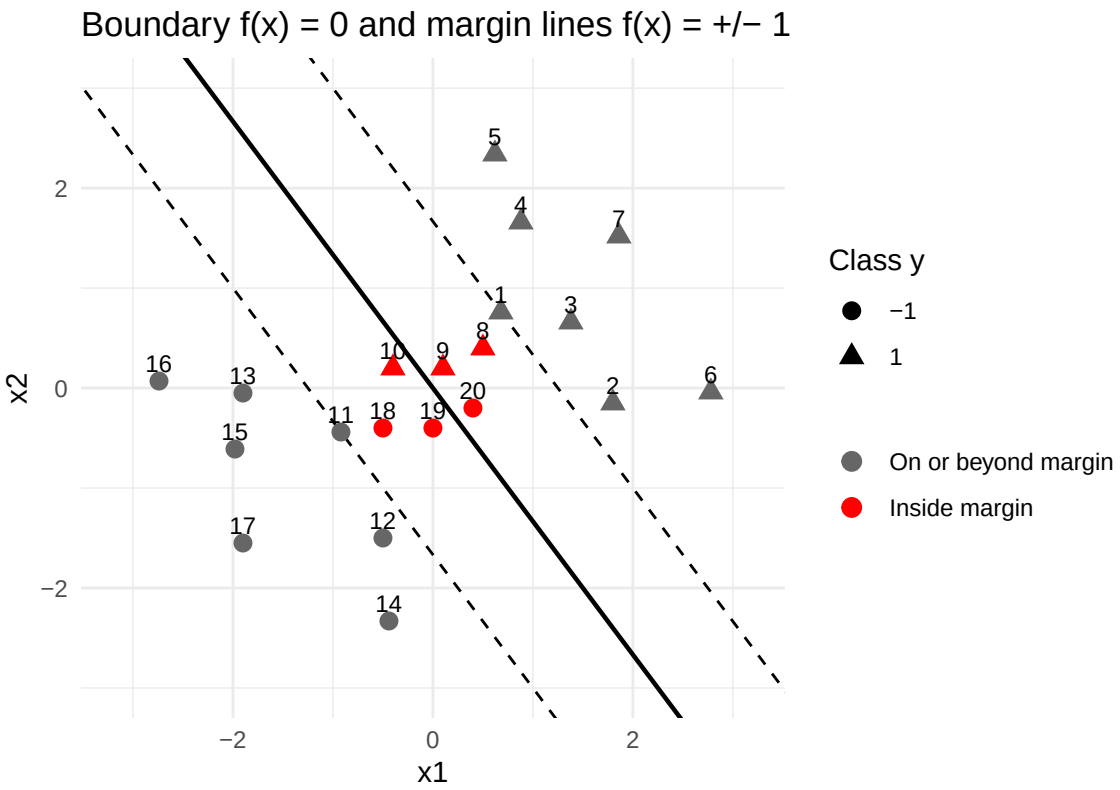
Observations 1 and 11 sit exactly on the margin (yf(x) = 1). Observations 8, 9, 10, 18, 19 and 20 have yf(x) below 1, so they are inside the margin. The other 12 have yf(x) above 1 and are beyond the margin.

b. Plot the observations, the boundary $f(x) = 0$, and the margin lines $f(x) = \pm 1$. Distinguish the two classes and identify the observations inside the margin in your plot.

```
slope <- -w[1] / w[2]

p1 <- ggplot(d1, aes(x1, x2)) +
  geom_abline(slope = slope, intercept = 0, linewidth = 0.8) +
  geom_abline(slope = slope, intercept = 1 / w[2], linetype = "dashed") +
  geom_abline(slope = slope, intercept = -1 / w[2], linetype = "dashed") +
  geom_point(aes(shape = factor(y), color = status == "inside"), size = 3) +
  geom_text(aes(label = observation), nudge_y = 0.18, size = 3) +
  scale_color_manual(values = c("FALSE" = "grey40", "TRUE" = "red"),
                     labels = c("On or beyond margin", "Inside margin")) +
  scale_shape_manual(values = c("-1" = 16, "1" = 17)) +
  coord_fixed(xlim = c(-3.2, 3.2), ylim = c(-3, 3)) +
  labs(shape = "Class y", color = NULL,
       title = "Boundary f(x) = 0 and margin lines f(x) = +/- 1") +
  theme_minimal()

p1
```




```
ggsave("figures/margin-geometry-20.png", p1, width = 6, height = 5.5, dpi = 300)
```

The solid line is the decision boundary and the dashed lines are f(x) = 1 and f(x) = -1. Triangles are class 1 and circles are class -1. Red points are inside the margin, and every point is labeled with its observation number. Observations 1 and 11 fall exactly on the dashed lines.

c. Report how many observations are inside, on, and beyond the margin. Which observations are misclassified by this decision rule?

```
d1 |> count(status) |> kable()
```

| status | n |
|---|---|
| beyond | 12 |
| inside | 6 |
| on | 2 |

```
d1 |> filter(misclassified) |>
  select(observation, x1, x2, y, f, margin) |>
  kable(digits = 3)
```

| observation | x1 | x2 | y | f | margin |
|---|---|---|---|---|---|
| 10 | -0.4 | 0.2 | 1 | -0.2 | -0.2 |
| 20 | 0.4 | -0.2 | -1 | 0.2 | -0.2 |

Of the 20 observations, 6 are inside the margin, 2 are on the margin, and 12 are beyond it. The misclassified observations are 10 and 20. Their yf(x) is negative, which means the sign of f(x) disagrees with the label. Misclassified points are also counted as inside the margin, since a negative functional margin is less than 1.

Use the observation numbers supplied in the data. Set the $x_1$ axis to $[-3.2, 3.2]$, set the $x_2$ axis to $[-3, 3]$, and use an equal aspect ratio. Do not fit a new classifier for this question.

## Question 2: Margin in four dimensions

Use `margin-geometry-4d.csv` from the combined ZIP, which contains 50 observations with four predictors and class labels $y \in \{-1, 1\}$. Consider the linear score

$$f(\mathbf{x}) = \beta_0 + \mathbf{x}^\mathsf{T}\beta, \qquad \beta_0 = -1, \quad \beta = (1, 2, -2, 4)^\mathsf{T}.$$

```
d2 <- read_csv("data/margin-geometry-4d.csv")

X <- as.matrix(d2[, c("x1", "x2", "x3", "x4")])
beta0 <- -1
beta <- c(1, 2, -2, 4)
norm_beta <- sqrt(sum(beta^2))

d2 <- d2 |>
  mutate(
    f = beta0 + as.vector(X %*% beta),
    proj = as.vector(X %*% beta) / norm_beta,
    fmargin = y * f
  )

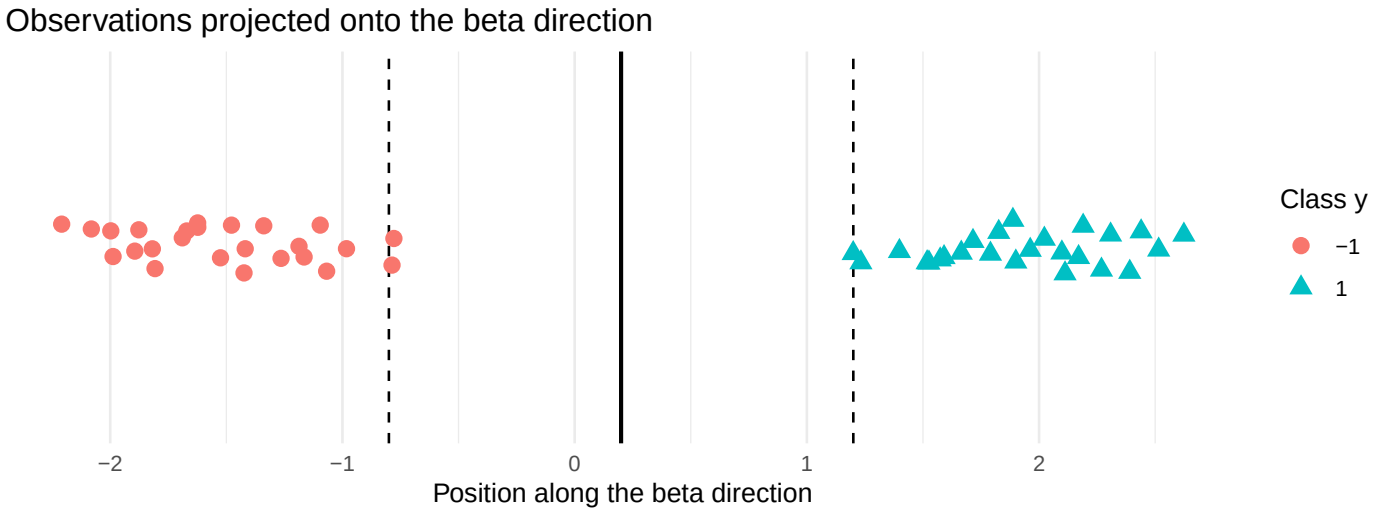
boundary_pos <- -beta0 / norm_beta
M <- min(d2$fmargin) / norm_beta
```

a. Project the observations onto the direction of $\beta$ and plot their positions on one axis. Distinguish the two classes and mark the decision boundary. Use the plot to calculate the geometric margin $M$, the distance from the boundary to the closest observation.

Projecting onto the unit vector beta / ||beta|| puts each observation at the position x'beta / ||beta|| on one axis. The norm of beta is sqrt(1 + 4 + 4 + 16) = 5. The boundary f(x) = 0 is where x'beta = 1, so on this axis it sits at 1/5 = 0.2.

```
p2 <- ggplot(d2, aes(x = proj, y = 0, color = factor(y), shape = factor(y))) +
  geom_vline(xintercept = boundary_pos, linewidth = 0.8) +
  geom_vline(xintercept = boundary_pos + c(-M, M), linetype = "dashed") +
  geom_point(size = 3, position = position_jitter(height = 0.05, seed = 1)) +
  scale_y_continuous(limits = c(-0.3, 0.3), breaks = NULL) +
  labs(x = "Position along the beta direction", y = NULL,
       color = "Class y", shape = "Class y",
       title = "Observations projected onto the beta direction") +
  theme_minimal()

p2
```



```
ggsave("figures/margin-geometry-4d-projection.png", p2,
       width = 8, height = 3, dpi = 300)
```

```
tibble(
  quantity = c("||beta||", "Boundary position", "Geometric margin M"),
  value = c(norm_beta, boundary_pos, M)
) |> kable(digits = 4)
```

| quantity | value |
|---|---|
| ||beta|| | 5.0 |
| Boundary position | 0.2 |
| Geometric margin M | 1.0 |

The solid line at 0.2 is the decision boundary. The closest class 1 observations project to 1.2, and the closest class -1 observations project to -0.8. Both are exactly 1 unit from the boundary, which is where the dashed lines are drawn. So the geometric margin is M = 1. Equivalently, the smallest value of y f(x) is 5, and 5 / ||beta|| = 5 / 5 = 1.

b. Express the same decision boundary using the SVM normalization in which the smallest functional margin $y_i f(\mathbf{x}_i)$ is set to 1. What are the corresponding values of $\beta$ and $\beta_0$?

```
m <- min(d2$fmargin)
beta_svm <- beta / m
beta0_svm <- beta0 / m

tibble(
  parameter = c("beta0", "beta1", "beta2", "beta3", "beta4"),
  value = c(beta0_svm, beta_svm)
) |> kable(digits = 4)
```

| parameter | value |
|---|---|
| beta0 | -0.2 |
| beta1 | 0.2 |
| beta2 | 0.4 |
| beta3 | -0.4 |
| beta4 | 0.8 |

```
# checks
min(d2$y * (beta0_svm + as.vector(X %*% beta_svm)))  # should be 1
```

```
[1] 1
```

```
1 / sqrt(sum(beta_svm^2))                            # should equal M
```

```
[1] 1
```

The smallest functional margin with the original coefficients is 5, so dividing beta0 and beta by 5 makes it exactly 1 without moving the boundary. The SVM-normalized values are beta0 = -0.2 and beta = (0.2, 0.4, -0.4, 0.8). The norm of this beta is 1, so the geometric margin is M = 1 / ||beta|| = 1, matching part (a). ## Question 3: Peer review of a Homework 7 skill

The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 7 submission. It will link to **one** assigned student's repository.

the link: https://github.com/amoghu26/stat432-fall2026/issues/6

So when I ran your skill, I noticed a few things! In my run the agent produced a report with PCA-LDA (6 PCs) with CV error 0.2741 and test error 0.2464 (34/138), seed 2026. In my run the agent produced a complete, readable one-page

report with a figure, table, and numerical results, selecting PCA-LDA (6 PCs) with CV error 0.2741 and test error 0.2464 (34/138), seed 2026. The run took abot 5 minutes, used about 87.2k/1M tokens, and finished without needing a steer message. The main weaknesses are test-set discipline, under-specified choices that make results hard to reproduce, and a report that repeats information instead of adding it.

Item 1: Test-set wording and leakage

WHat I found: The skill's report section says to highlight the "test-selected model" in the figure, but the procedure selects by CV. Your report also quotes plain LDA's test error (0.2826), although the table shows a test error only for the selected model and the workflow says to evaluate once. In addition, the report says the split was stratified, but the code shown shuffles with a permutation and slices, which is not stratified. This is impportant because you are computing test error for a non-selected model is a second look at the test set, even if the choice did not change. A literal-minded agent could also highlight the model with the best test error, which defeats the purpose of reserving it. My recommendation is that you change the wording to "CV-selected model." Add an explicit rule that test error is computed for the selected model only and never reported for any other candidate. Require the report to state the class counts in the training and test sets, so the split can be checked against the description.

Item 2: Under-specified choices hurt reproducibility

What I noticed: The skill says "chosen number of components" and "a simple average" but gives no default grids, no rule for choosing the seed, no instruction to reuse the same folds for every candidate, and no tie-breaking rule. My agent tried 2/4/6/8 PCs and blend weights 0.25/0.50/0.75, while the author's run used 3/5/7 PCs and a single 0.5 blend. The selected models differ (PCA-LDA with 6 PCs versus PCA-QDA with 5 PCs). Two blends in my run tied exactly at 0.3024 CV error with no rule for choosing between them. This is important because two agents following the same skill can reach different conclusions, so the skill does not make the analysis repeatable. Using the same folds is also needed for a fair comparison. I recommend you addd default grids (for example PCs from 2 up to the number of predictors, and a small set of blend weights), require one fixed seed and one set of folds shared by all candidates, state that predictors are standardized before PCA, and add a tie-break rule such as preferring the simpler model or using the one-standard-error rule. Also say what to do when a class covariance matrix is singular.

Item 3: The report repeats itself and lacks uncertainty

Observed: The bar chart and the table show the same CV errors, and the table's test-error column is empty except for one row. The "numerical model-fitting results" are only the CV and test errors. There is no spread across folds, no confusion matrix, and no baseline. The top candidates (0.2741, 0.2776, 0.2871) differ by less than 0.015, and the test estimate of 0.2464 from 138 cases has a binomial standard error of about 0.037. A majority-class rule would have a test error of about 0.348 (48/138), but the report never says so. This matters since a reader cannot tell whether the selected model is meaningfully better than its competitors or than doing nothing, and the one-page limit is spent on duplicated content. I would say you require the table to show CV mean with its standard error, and the figure to show something different, such as CV error with error bars against the number of components. Require the selected model's test confusion matrix, the majority-class baseline error, and an approximate interval for the test error.